

Using POGIL to Help Students Learn to Program

HELEN H. HU and TRICIA D. SHEPHERD, Westminster College

POGIL has been successfully implemented in a scientific computing course to teach science students how to program in Python. Following POGIL guidelines, the authors have developed guided inquiry activities that lead student teams to discover and understand programming concepts. With each iteration of the scientific computing course, the authors have refined the activities and learned how to better adapt POGIL for the computer science classroom. This article details how POGIL activities differ from both traditional computer science labs and other active-learning pedagogies. Background is provided on POGIL's effectiveness. The article then includes a full description of how POGIL activities were used in the scientific computing course, as well as an example POGIL activity on recursion. Discussion is provided on how to facilitate and develop POGIL activities. Quotes from student evaluations and an assessment on how well students learned to program are provided.

Categories and Subject Descriptors: K.3.2 [Computer Milieux]: Computers and Education- Computer and Information Science Education [Computer science education]

General Terms: Human Factors

Additional Key Words and Phrases: POGIL, active learning, process-oriented guided inquiry learning, inquiry-based learning, process skills

ACM Reference Format:

Hu, H. H. and Shepherd, T. D. 2013. Using POGIL to help students learn to program. *ACM Trans. Comput. Educ.* 13, 3, Article 13 (August 2013), 23 pages.

DOI: <http://dx.doi.org/10.1145/2499947.2499950>

1. INTRODUCTION

Process-Oriented Guided Inquiry Learning, or POGIL, is a student-centered learning approach that focuses on concept development in the framework of learning teams. Instead of passively listening to a traditional lecture, students work together in groups on specifically designed activities that guide students through the construction of course content (the *guided inquiry learning* of POGIL). When students discover concepts for themselves, they remember them better. In addition, this active learning approach emphasizes student development of key process skills, such as critical thinking, problem solving, teamwork, and leadership skills (the *process-oriented* focus of POGIL). This research-based pedagogy has been demonstrated highly effective in high school and college chemistry classrooms across the country [Moog and Spencer 2008].

POGIL was originally developed and used in college general chemistry courses in the mid-1990's [Farrell et al. 1999]. Since that time POGIL materials have been developed for biochemistry, organic, and physical chemistry and have been incorporated in a variety of settings including classrooms and laboratories at high schools, small colleges, and

Authors' addresses: H. H. Hu (corresponding author), Computer Science Department, Westminster College, Salt Lake City, UT, 84105; email: hhu@westminstercollege.edu; T. D. Shepherd, Chemistry Department, Westminster College, Salt Lake City, UT, 84105.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2013 ACM 1946-6226/2013/08-ART13 \$15.00

DOI: <http://dx.doi.org/10.1145/2499947.2499950>

large universities. More recently, POGIL has been introduced in biology, engineering, mathematics, and computer science classes. Much of this widespread expansion can be attributed to the POGIL Project, which seeks to provide structured support for the dissemination, implementation, and assessment of POGIL and other student-centered pedagogies [Moog et al. 2008].

While there is no single way to incorporate POGIL in a classroom, a typical implementation will involve:

- students working in self-managed learning teams;
- the use of specially designed guided inquiry materials; and
- the instructor facilitating the development of process skills.

The first feature ensures the active nature of the learning environment. Students work in small groups, with each student assigned a unique role such that all students fully participate in the learning process. For example, the responsibility of the “manager” is to keep the team focused and make sure everyone participates and understands the activity, while the “spokesperson” presents the team’s solution to the rest of the class. The second element of the POGIL methodology is the materials used to guide higher-level thinking and learning. A successful POGIL activity guides students through a series of questions to construct an understanding of the material for themselves, rather than providing an explanation of the material in lecture or written format. Finally, the third characteristic of this approach is the shift in the role of the instructor from disseminator of information to facilitator of the learning process. Rather than always serving as the direct source of information, the instructor continually assesses how and when to offer additional guidance as the students work together to construct knowledge of course content.

Over the past six years, the authors of this article have codeveloped an introductory programming class for science majors. The authors have created 24 original guided inquiry lessons designed to lead students through the process of writing, testing, and analyzing programs while simultaneously incorporating scientific applications. Each lesson follows the same basic structure: identify the scientific context, review the fundamental math/science principles used in the program, and introduce a specific programming method or algorithm to address the science. Students worked in teams as they learned to program using Python. With each iteration of the scientific computing course, the authors have refined the POGIL activities to improve their efficacy in the computer science classroom.

The background section of this article presents research supporting POGIL’s effectiveness in the classroom and details how POGIL activities have been used in computer science education. The next section describes unique features of the POGIL methodology that differ from other active-learning pedagogies used in computer science. Section 4 provides an overview of the scientific computing course and describes how the authors have incorporated three characteristic components of POGIL: learning teams, guided inquiry, and process skill development. The article then presents an example POGIL activity on recursion and includes commentary related to the specific POGIL materials used in the course. Section 6 provides data from the scientific computing class, including a discussion of the ways in which the authors have modified the class with each course offering. A comparison is made between the scientific computing class (cotaught by both authors using POGIL) and a CS1 class (taught by the first author with traditional lectures and computer labs). The article concludes with some POGIL resources for anyone who wishes to learn more about using POGIL in their classes.

Table I. Student Grades from General Chemistry Classes at Franklin and Marshall College

Grade	Lecture (n = 420)	POGIL (n = 438)
A	19.3%	24.2%
B	33.1%	40.6%
C	25.7%	25.6%
D, F, Withdraw	21.9%	9.6%

Source: [Farrell et al. 1999].

Table II. Common Exam Grade Distributions for Organic Chemistry Classes at a Large University

Grade	Lecture (n = 109)	POGIL (n = 75)
A	12%	9%
B	19%	32%
C	16%	31%
D, F or W	53%	28%

Source: [Straumanis and Simons 2008].

2. SURVEY OF POGIL IMPLEMENTATION AND EFFECTIVENESS

The fundamental research base of the POGIL methodology is rooted in ideas that have been advocated in a variety of contexts since the late 1950's. An overview of this literature foundation is presented elsewhere [Moog and Spencer 2008]. In addition, a number of studies have been conducted that specifically investigate the impact of POGIL materials on student learning. While most of the education research on the effectiveness of POGIL has been conducted in chemistry classrooms, a variety of different implementation contexts have been explored. For example, some instructors have chosen to replace all their lectures with POGIL activities [Farrell et al. 1999], while others retained all lectures but used POGIL activities during recitation sessions [Hanson and Wolfskill 2000]. Others have reduced the lecture to only 1/3 of the class time with new concepts and topics introduced using the POGIL materials the rest of the time [Ruder and Hunnicutt 2008].

2.1. POGIL in Chemistry

One of the most common outcomes from studies of student performance in a POGIL classroom is a decrease in the number of students who did not pass the course (whether by withdrawing or receiving a D or F grade). Moog, Farrell, and Spencer conducted an eight-year study of their general chemistry classes at Franklin and Marshall College. Each section consisted of approximately 25 students. From Fall 1990 until Spring 1994, these sections were taught by lecture. In all, 78% of the 420 students passed the class with a grade of an A, B, or C. Starting in Fall 1994 until Fall 1997, the classes were converted to POGIL classrooms, where 90% of the 438 students passed [Farrell et al. 1999] (see Table I).

In another study conducted at a large public research university, students were randomly placed into one of two sections of organic chemistry. One section was taught by lecture, while the other section followed POGIL pedagogy. Different instructors taught the two sections, but the same multiple-choice final exam (written by the lecture instructor) was given to both sections. A much higher percentage of students in the section using POGIL (72%) passed the final with an A, B, or C relative to the students in the lecture section (47%) [Straumanis and Simons 2008] (see Table II).

Straumanis and Simons also examined responses by 1027 organic chemistry students at six institutions to the Student Assessment of Learned Gains (SALG) to determine if POGIL helps students develop process skills. POGIL students reported significantly more growth in POGIL process skills (e.g., working with others, problem solving) than the lecture students. In addition, significantly more POGIL students said they “would recommend the method of teaching used in this course to a student taking the course next year” (75% compared to 54% of the lecture students) [Straumanis and Simons 2008].

Another POGIL study demonstrated better retention of content past the end of the course. At Virginia Commonwealth University, Organic Chemistry I was taught in two sections: a POGIL section and a traditional lecture section. At the start of Organic Chemistry II, all students were given an unannounced quiz on Organic I material. Over 20% of the POGIL students (but none of the lecture students) scored above 90% on the quiz. Most lecture students scored under 50%, whereas fewer than a quarter of the POGIL students scored similarly low [Ruder and Hunnicutt 2008].

2.2. POGIL in Computer Science Classrooms

Kussmaul has introduced POGIL pedagogy to several computer science classes, including CS2, software engineering, and a graduate-level class on neural networks and fuzzy systems [Kussmaul 2011; Hu and Kussmaul 2012]. Other computer science faculty report they have been using activities “structured like POGIL” in their introductory computer science classes [Coleman and Lang 2012].

Kussmaul [2012] believes POGIL is especially useful for teaching computer science students important problem-solving and team process skills. Because the biggest challenge to incorporating POGIL activities in more computer science classrooms is the lack of activities, he is collecting and sharing POGIL materials online at <http://cspogil.org>. As part of a NSF funded project, Kussmaul is creating two sets of 8-10 POGIL activities for computer science classrooms, which should be available online by 2014.

3. POGIL'S DEFINING ATTRIBUTES AND RELATED WORK IN COMPUTER SCIENCE

As stated in the Introduction, there are three fundamental features of the POGIL methodology: (1) students are organized into learning teams, (2) they work on guided inquiry-based activities, and (3) the instructor facilitates an environment that promotes the development of process skills. While many computer science classes require group work, fewer classrooms have combined collaborative learning with guided-inquiry learning, especially where the activities have been designed with the learning process in mind, rather than just the end result. In the remaining part of this section, each of these three aspects is discussed in the context of POGIL and compared to similar student-centered learning techniques currently used in computer science classrooms.

3.1. Learning Teams

In a POGIL classroom, the assignment and management of the learning teams is very dependent on the size of the class. For smaller class sizes, the instructor is able to specifically assign groups and closely monitor roles and interpersonal dynamics. In this setting the instructor is able to continually assess the cooperative effort of the group and encourage those who tend to work independently to improve their teamwork skills. Specific roles are assigned to each student to foster interdependence as well as individual responsibility and accountability to the success of the team. In a larger classroom, students tend to work in self-assembled groups based on their proximity in the room. Typical POGIL roles are described in detail in Table III.

A variety of strategies have been used to successfully facilitate the teamwork of students in this setting [Yeziarski et al. 2008].

Table III. Typical POGIL Roles for Students

Role	Responsibilities
Manager	Keeps track of time, makes sure all group members are contributing, and considers how the team might work more effectively together
Speaker	Serves as the team representative when speaking to the facilitator and other teams.
Technician	Types at the computer and runs programs.
Recorder	Records answers to questions and provides copies at the end of the activity to all team members and to the facilitator.

Pair programming is a popular practice in computer science pairing two students at one computer. Although this team approach involves fewer number of students than a typical POGIL group of 3 or 4, it is similar to POGIL in the assignment of roles. While one student, the *driver*, directly types the code, the second student serves as the *navigator*, correcting typos and other errors while thinking ahead about the algorithm. Students are prompted to switch roles regularly during a class period (whereas POGIL teams are prompted to switch roles at the start of a new class period). There are numerous studies finding that students who pair program enjoy their experience more, learn more programming skills, and pass their first computer science course at higher rates [Braught et al. 2011]. Pair programming also encourages students to practice and improve their teamwork, collaboration, and communication skills [Williams 2007].

Similar to how POGIL activities might be facilitated in a large lecture setting, Peer Instruction (PI) often relies on students to form their own groups with students sitting in close proximity. Simon et al. [2010] used PI in their introductory computer science course, by asking an average of four clicker questions per lecture. After individually answering each clicker question, Simon's students were encouraged to discuss the question with their peers. After three to five minutes of discussion, students answered the same clicker question again, and a class-wide discussion would take place before the instructor would confirm the correct answer. Students did vote more correctly after the discussion, and surveys showed most students found PI valuable to their learning. However, a surprisingly large number rarely or only sometimes participated in the discussion, which Simon plans to address in future classes by assigning discussion groups of four students each. It is interesting to note that Simon found that creating difficult but meaningful clicker questions was time consuming but imperative for a successful PI experience. This is similar to the authors' experience creating meaningful POGIL activities.

Carter [2012] has applied a hybrid of PI and Just-in-Time Teaching (JiT) to an introductory computer course for engineering students. At the start of each class, students were tested with clicker technology on assigned reading and online viewing materials. If an insufficient number of students answered the clicker questions correctly, mini-lectures and/or peer instruction would take place. Groups of two to three students spent the remaining time in class working through a worksheet of learning activities, which included "a mix of multiple-choice, short answer, code reading, and code writing questions," while the instructor and TA circulated the room. Carter's hybrid class structure is similar to Team-Based Learning (TBL), where students are also assessed on prereading, lectured to on difficult concepts (based on the assessment at the start of class), and grouped together to work on exercises that apply the concepts [Lasserre 2009]. Whereas both JiTT and TBL test students on what they have learned from assigned readings and online screencasts, preactivity questions used in POGIL pedagogy are intended to only *review* background material, particularly in advanced courses with prerequisites.

POGIL pedagogy discourages pre-reading, preferring instead that students encounter and explore content material for the first time in a guided-inquiry activity and later follow-up with post-reading. Because students typically accept information as fact from a textbook, the very nature of first reading the text prior to starting a POGIL activity circumvents the exploration and concept invention phase of the learning cycle. If “conclusions” are already known prior to the activity, the process of students “inventing” the knowledge is lost. This is not to say that a mature student cannot augment her knowledge, by “inventing” concepts previously learned. Most students, however, will simply take the path of least resistance and assume they already “know” the answer. This commonly results in students’ inability to apply the underlying concept to a new situation because they have no ownership of the idea.

POGIL is perhaps most often compared with Problem-Based Learning (PBL) and Peer-Led Team Learning (PLTL). PBL usually organizes students in larger groups and assigns more complex and open-ended problems with less formal guidance than POGIL activities [Greening et al. 1997]. Both POGIL and PBL activities are typically used during regular class periods in place of lectures. In contrast, PLTL sessions are typically held outside normal class hours. PLTL differs significantly from POGIL in that a more experienced student (someone who has recently completed the course), rather than the instructor, serves as the facilitator, leading students in activities and discussions to improve their understanding of the material [Huss-Lederman et al. 2008]. In addition, both PBL and PLTL typically assign students to permanent groups for the entire semester, whereas POGIL students are often grouped into new learning teams repeatedly through a semester. Eberlein et al. [2008] provide a thorough comparison on the similarities and differences among PBL, POGIL, and PLTL.

3.2. Guided Inquiry

The most defining feature of POGIL is the distinct design of the materials used in the classroom. The activities are written such that students rely on the team approach to answer the questions. Since the questions are designed to guide students through the construction of new knowledge, they rely on the confirmation they receive from their group members that they are “correctly” answering the questions. The materials typically follow a learning cycle approach of *exploration*, *concept invention*, and *application*. During the exploration stage, the student is guided to consider various aspects of the “model” or information given that serves as a basis of the new understanding that will be constructed. During the concept invention phase, the critical thinking questions prompt students to discover patterns and develop concepts typically before the standard terms for these ideas are introduced. Finally, the concept is reinforced and extended in the application phase with exercises and problems. Abraham [2005] provides a more thorough explanation of this three-stage learning cycle. The application and development of at least one process skill is embedded in the structure or content of the POGIL activity. These characteristics are vital to the development of higher-level thinking skills and the ability to learn and apply discipline-specific knowledge in new contexts.

Many computer science educators have noted that inquiry-based learning is especially appropriate for computer science classrooms [Abernethy et al. 1998; Kussmaul 2012]. Many educators are using inquiry-based learning activities to generate interest in computer science among high school students, who often find student-centered learning different and exciting. Since 2006, IT Adventures, an inquiry-based approach, has been teaching Iowa high school students about several IT topics, including programming using Alice. Each spring, many of its IT clubs’ participants compete in the IT-Olympics, where the students are also exposed to IT careers and educational opportunities [Rursch et al. 2010]. Goode and Chapman [2009] developed and implemented

an inquiry-based curriculum, *Exploring Computer Science*, in Los Angeles high schools to introduce students to computational thinking and computer programming. Subsequent research found that providing high school teachers with additional instructional coaching on inquiry-based teaching significantly increased the teachers' adoption of inquiry-based strategies in their classrooms.

Guided-inquiry learning has also been used successfully in post-secondary classrooms to teach computer programming. Case Western Reserve University professors developed a new inquiry-based course, Dynamics of Biological Systems, in which biology students programmed and modeled systems with Mathematica. The instructors credit the new inquiry-based module with improving student learning [Chiel et al. 2010, 2012]. Computer science professors at Furman University used inquiry-based learning in five classes, including CS1 and CS2. Students completed their pre-class inquiry-based assignments outside of class, working alone. The instructors experienced difficulty in creating meaningful problem-based questions (as opposed to syntax-based questions) for CS1, but had better success creating inquiry-based assignments on sorting algorithms for CS2 [Abernethy et al. 1998]. Medley [2007] simultaneously taught two sections of CS1: one traditional section and one using inquiry-based learning pedagogy. While some students chose to work in groups, groups were not assigned. Most of class time was spent on students presenting their answers to the problems, rather than working on the problems in class. Medley's inquiry-based section had higher withdrawal rates, but also a higher pass rate than the traditional course.

Computer science instructors have found success engaging students and transferring knowledge to students by using inquiry-based activities. The difficulty in developing well-designed inquiry-based activities is perhaps the largest obstacle to wider use of guided-inquiry pedagogy. Just as POGIL activities are being collected by the POGIL CS project (<http://cspogil.org>), the Exploring CS project (<http://www.exploringcs.org/curriculum>) and the Dynamics of Biological Systems course (<http://biol300.case.edu/>) have also shared their curricula online.

3.3. Process Skills

Both POGIL materials and the instructor's classroom facilitation plan are intended to develop and improve students' skills for critical thinking, problem solving, communication, teamwork, leadership, and assessment. Acquiring these process skills are considered as important as learning the traditional content of the class (e.g., how to write a computer program). Some of the process skills, such as communication, teamwork, and leadership, are developed by assigning students roles in their learning teams. The guided-inquiry learning promotes the development of problem-solving skills. Students are asked to identify their insights during problem solving, which in turn improves their problem-solving skills. In another technique, every learning team is asked to simultaneously write their team's solutions on a board, so the students can work out disagreements together (enhancing learning, problem solving, communication, and assessment skills). Many of these process-oriented instructional strategies are based on novice-expert research [Hanson 2006].

Others in computer science have also based their pedagogies on the same novice-expert research. Cognitive Apprenticeship (CA) uses "teaching by coaching" with the stages: modeling, scaffolding, fading, articulation, reflection, and application [Bareiss and Radley 2010]. A derivative of CA, eXtreme Apprenticeship (XA), emphasizes guided programming exercises in the modeling, scaffolding, and fading stages [Vihavainen et al. 2011]. As with POGIL, these apprenticeship pedagogies recognize that what skills might be learned during the *process* of solving a problem are often more important than the end product of creating a solution to the problem.

Still others have worked to teach specific skills to computer science students. Some classes have introduced Writing to Learn (WTL) activities to develop communication skills [Hoffman et al. 2006; Falkner and Falkner 2012], while others require students to make regular reflections or assessments of their software development process and thus become more metacognitive about their process [Fekete et al. 2000; VanDeGrift et al. 2011]. Many computer science programs aim to teach soft skills, like communication, teamwork, and organization, through service learning and/or capstone classes [Brown et al. 2009; Carter 2011].

A recent study makes recommendations on questions that might be asked by a computer science instructor or tutor facilitating student learning. seventy-eight tutoring sessions between untrained tutors and introductory computer science students were analyzed. The tutors' questions for the students were classified based on the instruction goal for asking the question and *realized* question type (e.g., hint, procedural question, concept completion question). Boyer et al. [2010] conclude that instructors and tutors could better achieve their instruction goals by encouraging comprehension and decomposition of a problem, eliciting self-explanations from the student, asking targeted questions, and asking questions frequently. Their paper includes examples of dialog that should be helpful to any instructor, including those facilitating POGIL activities.

While the Boyer et al. [2010] study offers advice specific to computer science, the POGIL project sponsors regional workshops to help instructors from any discipline become better facilitators. These workshops also provide guidance on how to develop a POGIL activity that develops one or more process skills and offer an excellent opportunity to meet other instructors with whom to share POGIL activities. Workshops are listed online at <http://www.pogil.org/events>.

4. SCIENTIFIC COMPUTING: A POGIL COURSE

Since 2005, the authors have cotaught an upper-division scientific computing class every other year at a small liberal arts college. A number of elements of this course are common to many CS1 courses. Students learn how to program, with an emphasis on problem solving and algorithm design. Students also learn to create dynamic computer simulation models, to interpret models' results, and to use models to develop an understanding of scientific processes. The course meets twice a week, two hours at a time. There is no required textbook, although there are online readings on Python. Physics majors are required to take this course; other science majors earn upper-division elective hours. The course requires one semester of calculus and one semester of physics as prerequisites, so students enrolled in the class are typically junior and senior science majors. Students were encouraged to use their own laptops, but could also use one of the science department's laptops instead. These MacBooks have Python installed and include network access to the students' computer accounts.

Prior to taking scientific computing, many of these students have experienced POGIL in one or more chemistry classes taught by the second author. Enrollment for each scientific computing class has never been more than 22 students. The following sections detail how each of the three characteristic features of POGIL were implemented in the course.

4.1. Learning Teams

Because every POGIL lesson in scientific computing involves Python computer programming, students were assigned to work side-by-side in pairs at one computer and across the table from another pair. Specific student pairs were reassigned frequently toward the beginning of the semester. As the semester progressed, the pairings became relatively static as the instructors worked out optimal groups of student personalities and abilities to maximize learning. Strong students were usually paired with other

strong students, and weaker students were paired with an average student. For this interdisciplinary course, some attempt was made to pair students with different majors to broaden the skill depth of each team. As with any learning environment that relies heavily on active student participation, tardiness and absences can lead to nonideal practices. While it sometimes worked out for pairs of tardy students to work together, it was the student's responsibility to complete any missed material on her own. Most experienced POGIL facilitators find attendance is better than that of lecture-based courses, because the active nature and team responsibility leads to improved class attendance.

Each student in a learning team was assigned one of two roles: manager/navigator and technician/driver. The manager is responsible for keeping the team on task, working together through the questions, spotting typos or syntax errors, and suggesting test cases. The technician types at the computer, writing and running the program. Students were prompted regularly to switch roles, usually at the start of each "model" in the lesson. Students were expected to write programs in pairs, but were prompted to discuss algorithms and problems in groups of four at their table.

While other computer science instructors have exclusively employed traditional POGIL roles in their classrooms [Kussmaul 2012], the instructors decided to use a hybrid approach, assigning pair programming roles for the explicit programming sections of the activity, but encouraging students to work in groups of four (two teams of two) when answering the critical thinking questions. For computationally intensive work, the students seemed to work best sharing one computer between two students. Larger groups of students had trouble positioning themselves around a single laptop so that everyone could see the screen clearly. But when students were prompted by critical thinking questions to discover and develop general programming concepts, they naturally tended to seek a broader consensus with nearby students. The instructors observed that specific roles were not needed to facilitate interactions of the larger group of four at each table. The students readily varied the tasks implied by the roles common to the POGIL approach. However, in a classroom of students with less POGIL experience, assigning traditional POGIL roles might improve the learning experience for these groups of four students (see Table III).

4.2. Guided Inquiry

To prepare for each scientific computing lesson, students answered preactivity questions due at the start of class. These questions typically addressed any scientific or mathematical background material used in the activity. Students might trace an algorithm by hand (e.g., applying three iterations of Newton's method) or identify the expected output for a program they would write later in the lesson (identifying the corresponding RNA sequence for a DNA sequence). At the start of class, the solutions were collected, and if necessary, the correct solution was explained.

In addition to the initial preactivity questions, a typical scientific computing POGIL lesson consists of approximately three models, each followed by a series of critical thinking questions and/or programming exercises to be completed as a team, and individual homework questions to be completed outside of class. Each model presents one concept, often demonstrated with a short segment of Python code. Students are then guided by critical thinking questions to explore the concept, before they apply this newfound knowledge in programming exercises. Section 5 of the article describes in detail an example POGIL activity on recursion.

As POGIL facilitators, the instructors guided students through this process of exploration, concept invention, and application. At the beginning of the semester, the instructors explained and reinforced how students were expected to work through the activity. Students were reminded to carefully examine the "model" in the activity before answering the subsequent critical thinking questions. Each group was directed

to read each question together, propose and discuss potential answers, and write the answer agreed upon by all team members before continuing to the next question. If a team had a question, an instructor would consider whether to ask a different question, direct students to consider their answer in context of a previous question, or simply provide additional information leading more directly to the answer. When it appeared that a learning team had an incorrect answer, the instructors considered whether to intervene or to wait and see if the team would resolve their inconsistency on their own. If a team completed all the POGIL activities before the end of class, then they would start on the individual homework questions.

4.3. Process Skills

Basic POGIL facilitation strategies were used throughout the course. In a POGIL classroom, the instructor's attention is divided between facilitating effective teamwork and monitoring each team's progression towards discovering course content. To this end, the instructors would review each lesson and create a facilitation plan outlining the lesson's objectives, structure, and activities. The instructors used the plan to specify both relevant content and process objectives that would be monitored throughout the class period. Upon the completion of each lesson, these objectives were explicitly communicated to the students both in the printed activity and on the course management website.

For each class period, the instructors had a facilitation plan, predicting: (1) the amount of time students should work in teams to complete various aspects of the activity and (2) the amount of time the class as a whole would discuss key principles through a variety of "reporting out" techniques. For example, the instructors might project the amount of time it would take most groups to complete the critical thinking questions following the first "model" and then plan to spend five minutes with each team's spokesperson sharing his/her group's response to a key question that would provide insight for the class as a whole. Developing a specific facilitation plan is key to establishing an efficient pace for the class such that the majority of students are able to progress through the desired content during the class period. Minderhout and Loertscher [2008] provide an excellent discussion on factors to consider when creating a facilitation plan.

5. POGIL ACTIVITIES FOR SCIENTIFIC COMPUTING

In the Fall of 2011, scientific computing students worked through 24 POGIL activities in 28 class meetings. The activities in the first half of the semester were designed to teach programming concepts while using scientific contexts to motivate the programming. These POGIL activities include the following (with the corresponding scientific context given in parentheses):

- Python scripts and functions (temperature conversion);
- user-defined functions (coordinate system conversion);
- branching (quadratic equation);
- while loops (bisection method);
- for loops (Newton's method);
- strings, lists, and dictionaries (DNA and RNA);
- file I/O (molecular coordinate files);
- object-oriented programming (planetary systems).

During the second half of the semester, students explored how computer programs can model a scientific system, bringing scientists to a better understanding of the underlying science. Several POGIL activities built up to each of the following:

- Monte Carlo simulations of a random walk of a particle;
- numerical integration of various equations of motion, and the error associated with the approximations;
- transformations between Cartesian and internal coordinate representations of molecules;
- molecular dynamics and visualization of 2D ideal gas.

While students continued to improve their programming skills in the second half of the semester, the emphasis was on creating dynamic computer simulation models and using these models to develop an integrated understanding of the basic biological, chemical, and physical processes. For an outline of all 24 POGIL activities see the Appendix to this article.

For the remainder of this section, the article will describe and provide one model of a representative example of a POGIL activity. The complete POGIL activity, which leads students to understand and apply basic principles related to recursive programming, and the syllabus for the course are available online at <http://cspogil.org>.

5.1. POGIL Recursion Lesson: General Model Descriptions

Before starting this POGIL activity, students would have completed preactivity questions that ultimately require students to write a factorial function using loops. In class, students are paired into groups and given the following models.

Model 1. This model introduces the concept of a base case and solving problems using the solution for a smaller, simpler problem. No code is introduced yet, although the critical thinking questions at the end of the model prompt students to consider how recursive method calls might be used to calculate a factorial. See what follows for the entire text for this model.

Model 2. This model introduces a recursive method with example Python code. The example code calculates factorial while printing intermediate values (before and after the recursive call). Students are instructed to type in and run the example code. Based on the output, they answer questions on the number of recursive calls. Students are asked to input a negative number, find the error in the example code, and fix the code. They are prompted by a series of questions to write out the recursive definition for a summation on paper, before writing a new Python function to calculate a summation recursively. Before continuing to the next model, they also write a function to calculate a summation iteratively. A final series of questions prompt students to consider how the iterative and recursive functions differ.

Model 3. This model introduces a simple recursive function with only a base case (an if but no else). The function takes one argument and prints numbers starting at that argument down to the number 1 (e.g., 5 4 3 2 1). Students are asked to modify the code so it prints numbers counting up instead of down (e.g., 1 2 3 4 5). A reflection question prompts them to assess their thinking process in finding this answer, which requires only changing the order of the print command and the recursive call.

At the end of three models, students are asked to complete an individual assignment with additional recursive programming exercises, one of which demonstrates a more efficient calculation for x^n using recursion than what could be programmed with iteration. This entire activity would be completed in one two-hour class period. Any portion unfinished at the end of class would be assigned as homework, due the next week.

5.2. POGIL Recursion Lesson: Model 1

The following text is taken directly from a POGIL activity on recursion.

MODEL 1

When faced with a large problem to solve, we might seek to use a solution to a smaller, simpler problem. If we repeatedly decompose the original problem into smaller simpler expressions, we will eventually identify the most simple or basic component that can't be broken down any further. This is referred to as the *base case*.

CRITICAL THINKING QUESTIONS

- (1) Consider two different ways to show how to calculate $4!$
 - (a) Write out all numbers that explicitly need to be multiplied
 $4! =$
 - (b) Write the expression using $3!$
 $4! =$
- (2) Write an expression similar to CTQ (1b) showing how each factorial can be calculated in terms of a "simpler" factorial.
 - (a) $10! =$
 - (b) $3! =$
 - (c) $2! =$
 - (d) $1! =$
- (3) Generalize your group's answer to CTQ (2) in terms of n to create an equation for factorial that would be true for all factorials except the base case.
 $n! =$
- (4) What would your group propose to be the *base case* of a factorial function? Record your group's justification for this answer.
- (5) Assume you have defined a function `factorial (n)` that returns $n!$. Show how you might convert your answer to CTQ (2a) to a Python assignment statement that uses an appropriate function call and argument to calculate $10!$ in terms of a "simpler" factorial.
- (6) Based on the generalization in CTQ (5), write another Python assignment statement that specifies an appropriate function call and argument to calculate $9!$ in terms of a "simpler" factorial.
- (7) Convert your answer to CTQ (3) to Python code that would calculate $n!$ for any value of n .
- (8) Assume you incorporated your group's answer to CTQ (7) as code executed within `factorial (n)`. How would this function call differ from function calls you have used in previous programs?
- (9) Is a loop necessary to calculate $10!$ based on your group's answers above? Describe your group's reasoning.
- (10) What type of programming structure (branching or looping) is required to differentiate the successive function calls from the base case?

5.3. POGIL Recursion Lesson: Commentary

The goal of the series of questions following the first model is for students to "discover" the concept of recursion by following the learning cycle approach described at the start of Section 3.2. The first four critical thinking questions guide students to explore the model: the idea of breaking down a problem into simpler components and defining a base case. The concept invention stage of the learning cycle is seen in the next three questions through which students construct an understanding that a function can call itself. The programming term "recursion" is not introduced until the next model after the students investigate the basic meaning of the concept and see its utility. The last three questions are more application based, guiding the students to recognize the similarities and differences between recursive functions and the iterative functions

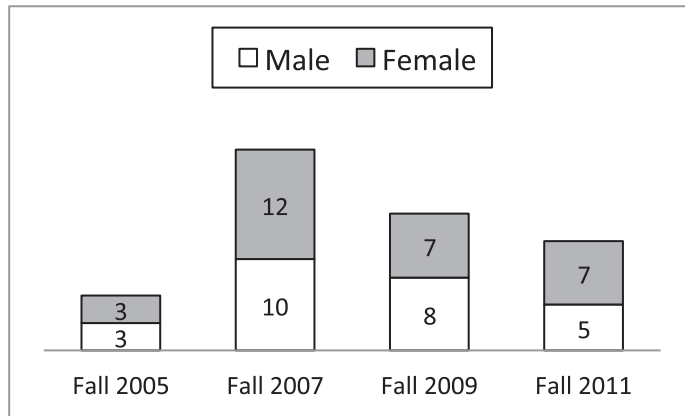


Fig. 1. Distribution of male and female students enrolled in scientific computing. Students have been fairly evenly divided by gender, with 52% female students across all four course offerings.

they have seen in previous lessons and to consider how a recursive function might be coded.

Embedded in the Critical Thinking Questions (CTQ) is the underlying assumption that students will be discussing and recording the answers to these questions as a group (e.g., CTQ (3), (4), (8), and (9)). Other process skills that are explicitly noted include communication (CTQ (9)) and problem solving (CTQ (3)).

This example lesson illustrates the distinguishing features that differentiate POGIL pedagogy from other student-centered instructional techniques. Instructors interested in authoring their own POGIL materials should refer to the POGIL Project website, which hosts many helpful documents on essential features for POGIL materials (<http://www.pogil.org/resources>). It is also possible to submit newly developed activities to the POGIL Project for feedback from experienced POGIL authors.

6. RESULTS

6.1. Class Composition

In all, 55 students have taken the scientific computing class since its first offering in 2005. Three of these students withdrew from the course before the college's drop deadline. Overall, 52% of the students have been female (Figure 1). The decrease in enrollment in 2009 and 2011 was partially due to scheduling conflicts. More students expressed interest in registering for scientific computing, but were disappointed when it was scheduled at the same time as another course required for their major.

The majority of the students have been chemistry majors (Figure 2). While scientific computing is required for physics majors, physics students who are dual math majors or computer science majors are able to substitute CS1 (a requirement for math and computer science majors). One of the strengths of this class has been the diverse group of students it has attracted across gender and major. Creating a valuable interdisciplinary experience for science majors is an important goal of this course.

6.2. Developing POGIL materials for Scientific Computing

The design and development of POGIL activities is a large time commitment for instructors. Because no POGIL computer science activities were available for the first offering of the scientific computing class in 2005, the authors ambitiously attempted to create POGIL lessons for the entire class in one semester. The combination of a content expert (first author) and seasoned POGIL author who had used POGIL exclusively in

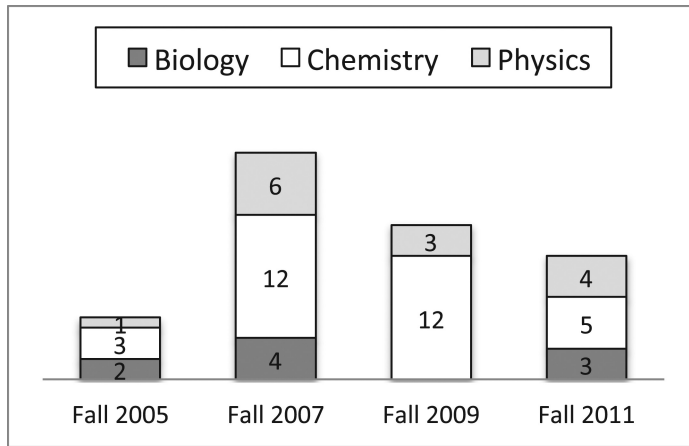


Fig. 2. Distribution of majors among students enrolled in scientific computing. The majority of enrolled students have been chemistry majors, followed by physics and finally biology majors.

other courses (second author) resulted in a very productive collaborative effort. Despite being able to share the workload, both authors agree the first semester was more labor intensive than either had anticipated. In the end, some later lessons were written more as traditional computer science labs rather than POGIL lessons. Based on their experience, the authors suggest other instructors only convert an entire course to POGIL pedagogy in one semester if some or most of the POGIL lessons are already available and tested. Otherwise, it may be more sensible to convert a few weeks to POGIL pedagogy at one time.

The authors have refined the models, critical thinking questions, and programming exercises based on their observations of students using the POGIL activities in the classroom. One insightful aspect of using POGIL is listening to the conversations of students as they are developing their knowledge of course content for the first time. Instructors will typically discover students' misunderstandings and misconceptions that they would never be aware of when delivering a lecture on the subject. The authors learned a lot over the years, particularly whenever multiple groups struggled with a concept or question. In the most recent offering of scientific computing (Fall 2011), the authors modified several POGIL activities based on the 2009 experience. In particular, the authors completely rewrote the POGIL activity on recursion. While the 2009 students had struggled with the activity and generally did not try to use recursion after the lesson, the 2011 students seemed to understand recursion better. Many of the 2011 students would try first to use recursion in later assignments, even when iteration would be a better choice.

The first version of many of the scientific computing POGIL activities emphasized learning through a series of developmental programming exercises, similar to the type of assignment that might be used in a traditional computer science lab. The addition of explicit "critical thinking question" guiding students to develop conceptual understanding of a particular algorithm resulted in some substantive organizational and content changes to the activities. These critical thinking questions are key to directing students' learning of new concepts.

The authors tried to limit most POGIL lessons to one class period, moving some models to new lessons while eliminating others that seemed redundant. At first glance, "deleting" content to provide more time for student discussion and discovery often leads to the misconception that one must sacrifice content when adopting the POGIL

approach. It is the experience of the authors, however, that a class using POGIL lessons can cover the same content as a traditional lecture-based class. The instructor may need to carefully identify the essential aspects of each topic, to recognize redundancies, and to differentiate between concepts that should be developed and reinforced during a class period and topics that can be further practiced and applied through homework.

While most changes in the POGIL materials over the years generally improved the class experience, undoubtedly some mistakes were made. For example, in 2011 the instructors eliminated one activity that reviewed looping, believing students should be able to use looping enough within the subsequent activities. A few weeks later, it became clear that the 2011 students were weaker at writing loops than the 2009 students. In future course offerings, the POGIL activity developing the concept and application of looping will be reworked.

Despite unavoidable missteps, by the end of the semester, the instructors were pleased with the overall progress made by the 2011 students. On the last day of class, both the 2009 and 2011 students worked through the same POGIL lesson on a simulation and visualization of gas molecules. The 2011 students produced significantly better-organized programs than the 2009 students. The instructors theorize that the improved reordering of POGIL models throughout the course and the enhanced lessons on object-oriented design may have contributed to the strong coding skills of the 2011 students.

The authors have shared three of their more generally applicable POGIL lessons with the POGIL in Computer Science project (<http://cspogil.org>). Many of the other lessons are Python specific or require more science background than the average CS1 course, but the authors are willing to share these with instructors who contact them. As more general lessons are improved, they will also be posted online.

6.3. Success Teaching Students to Program

The intent of this article is to communicate the authors' experience implementing POGIL to teach students to program. For the past seven years, much of their focus as instructors has been on writing and revising POGIL materials since this is the essential element of this teaching approach. The uniqueness of the course (i.e., a class for science majors, never taught previously in a traditional lecture format, with a heavy reliance on math and science prerequisites) limits the comparisons and analysis from which educational research-based conclusions can be drawn. Aside from a common assignment comparison between the one POGIL scientific computing class and a lecture-based CS1 class described in the next section, only typical evaluation results such as course grades and student evaluations can be used to assess the impact of this course on student learning.

Preactivity assignments were checked off quickly, often in class, and contributed to 5% of students' final grades. Answers to critical thinking questions were not graded, although instructors checked the answers for correctness as they circulated the room. Programs written together by learning teams (i.e., all the coding completed for the POGIL models and some homework programs) were 30% of the final grade. Programs written independently (the majority of the homework programs) were 25%. Two mid-term "quizzes" were 25% of the final grade. The final exam (15%) consisted of two portions: a traditional, closed-book written portion similar to the mid-terms and a take-home programming assignment.

Aside from the three students who withdrew from the course, most students successfully learn how to program and pass the course. The majority of students earn A's and B's (Figure 3). Students who earn C's, D's, or F's are often overcommitted; many are working full-time jobs in addition to carrying a full course load.

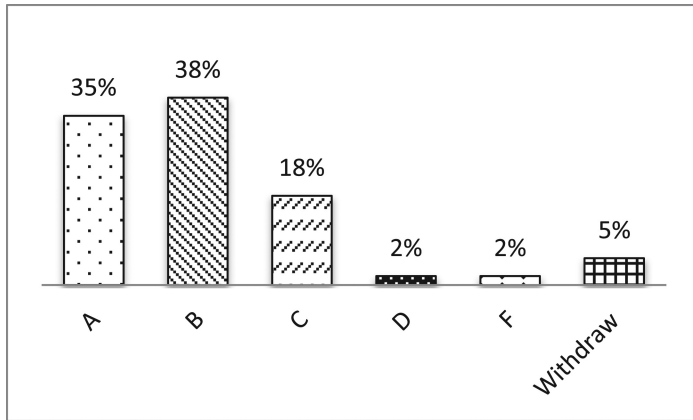


Fig. 3. Distribution of grades earned by all students who enrolled in scientific computing from 2005–2011.

For the past two course offerings, many of the comments from the end-of-course evaluations related to POGIL aspects of the course. Some students recognized the scaffolding of the activities: “It was great the way the assignments built on each other, slowly introducing new ideas.” (2011) Others commented on key process skills including group learning: “I did enjoy the learning structure, of both individualized and team work” (2009) and problem solving: “I felt like this introduced programming to me wonderfully. I feel like I might be able to apply programming in real-life situations now” (2009).

Another consistent observation of the second author is that a number of former students have voluntarily opted to write Python programs to solve homework problems in their upper-division science courses in place of more widely used mathematical software (including their scientific calculator). Other students create data analysis programs to use in their undergraduate research projects. In some cases, the students who were very proud to show off their programming skills after completing the course did not perform exceptionally well in class. This was quite surprising to the instructors, since it was assumed that these weaker programming students would probably never program again.

6.4. Comparison to a Traditional CS1 Course

While initially a POGIL activity appears to require more class time than a lecture on the same material, students typically master the concepts better and thus progress through more advanced material quicker than students in a traditional classroom. In the fall of 2007, the first author simultaneously taught the scientific computing class in Python and a traditional CS1 course in Java. The scientific computing course was POGIL based, whereas the CS1 course had traditional lectures and programming labs. The scientific computing class consisted of upper-class science majors, whereas the CS1 class was a mix of freshmen and sophomore computer science students and upper-class math majors. The POGIL class reached looping one week earlier than the traditional CS1 class, most likely because Python students had less overhead to learn than the Java students. The POGIL-based scientific computing course then spent more time on loops than the traditional CS1 course. Despite moving slower on basic programming concepts, the Scientific Computing students became proficient in programming larger programs at approximately the same time as the CS1 students, although some of the differences in time spent learning advanced topics may have related to differences in programming languages.

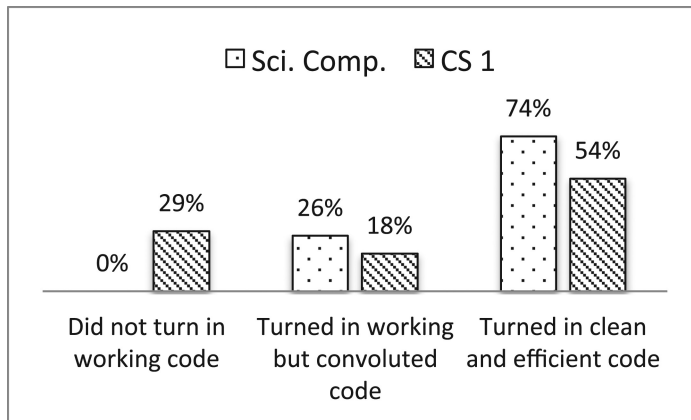


Fig. 4. Performance by scientific computing and CS1 students on a similar looping homework assignment to calculate a number from the Fibonacci sequence. All scientific computing students turned in working code, whereas only 71% CS1 students were successful. 74% of the scientific computing students and 54% of the CS1 students turned in clean and efficient code (as opposed to convoluted code suggesting programming by trial and error).

In September 2007, the scientific computing and CS1 students were given the same looping homework problem, due on the same day. Both sets of students were asked to write a program that calculated the n th value in a Fibonacci sequence, albeit in different programming languages. The rest of the homework assignments differed; both classes were given additional questions and programming problems to complete. Prior to the assignment, both classes were given simpler examples of loops in class. Figure 4 shows that all 19 scientific computing students turned in a working program (programs with minor off-by-one errors are counted among the “working” programs). (Two scientific computing students turned in working *recursive* solutions; they are not included in this comparison.) By contrast, 29% of the 28 CS1 students did not turn in a working Fibonacci program (ignoring off-by-one errors). This difference may partially be attributed to the freshmen CS1 students who had not yet developed college-level study skills, although more than one upper-class math major also did not complete the program.

Upon examination of their code, a few of the CS1 students and several of the scientific computing students turned in correct but somewhat convoluted code (i.e., code that was more complicated than strictly necessary). The authors theorize that these students may have been more persistent than the students who did not complete the assignment, but still did not have a full understanding of the solution. Instead, these more persistent students may have stumbled on a working program by trial and error. In total, 54% of the CS1 students and 74% of the scientific computing students turned in clean and efficient solutions. These results suggest that the scientific computing students learned to program with loops by working through POGIL activities just as well if not better than the traditional CS1 students.

The first author is planning a more controlled study of applying POGIL pedagogy in computer science. She has converted the POGIL lesson on recursion described in Section 5 from Python to Java. In Fall 2012, she plans to use the Java version in one of her CS1 sections. The two remaining sections will have a recursion lecture, one of which will additionally model recursion with a kinesthetic activity. She will compare how students from all three sections of CS1 perform on the homework assignments, quizzes, and test questions on recursion. The following semester, all CS2 students will be tested on the first day of class to assess how well they remember recursion.

7. CONCLUSION

POGIL's three characteristics distinguish it from other active-learning, student-centered techniques: the organization of learning teams, the use of guided-inquiry activities, and the emphasis on process skill development. Other fields, notably chemistry, have demonstrated students can learn concepts by working through POGIL materials instead of attending traditional lectures. The authors of this article have developed and refined POGIL materials for their scientific computing class. Most of their students have successfully learned to program by working through these POGIL activities.

The greatest obstacle preventing more computer science instructors from using POGIL pedagogies in their classes is the lack of POGIL materials in computer science. The CS-POGIL project is collecting POGIL activities in computer science to be shared at <http://cspogil.org> [Kussmaul 2012]. However, before more POGIL activities from this scientific computing class are disseminated to the general public, they must first be tested in multiple classrooms. The authors are eager to share all their POGIL activities with any instructor who is willing to provide feedback. Interested teachers should contact the authors for more information.

Ideally, an instructor should learn more about POGIL before attempting to facilitate a POGIL activity in the classroom. The POGIL Instructor's Guide [Hanson 2006] and other resources on implementing POGIL techniques are available online at the official POGIL Project website (<http://www.pogil.org/resources>). Educators who are interested in learning more about the benefits of this approach and experiencing a POGIL-based learning environment are strongly encouraged to attend a workshop organized through the POGIL project. The website also lists POGIL regional workshops, which are currently held every summer.

APPENDIX

This appendix provides a short description of 24 POGIL activities used in the described scientific computing course. The 24 POGIL activities were organized in four units: Programming Basics, Programming Design, Stochastic Simulations, and Deterministic Simulations. A more detailed description, listing every model in each POGIL activity, is available online at <http://cspogil.org>.

Unit I. Programming Basics

Lesson 1: Computational Focus: Introduction to Python

Scientific Context: Unit Conversions

Physical Fundamentals: Temperature Scales

Learning Outcomes:

- Identify the three variable types corresponding to integers, floating-point numbers, and characters.
- Write assignment statements using appropriately named identifiers.
- Call built-in Python functions with arguments.

Lessons 2 & 3:

Scientific Context: Coordinate Systems

Physical Fundamentals: Polar Coordinates

Lesson 2: Computational Focus: Writing Useful Functions Learning Outcomes:

- Define and execute a custom Python function.
- Use Python math module functions.
- Create a function that takes in variables as parameters and returns a value to be used outside the function.

Lesson 3: Computational Focus: Scope Learning Outcomes:

- Control variable type using explicit type cast.
- Reassign the value of a variable through an assignment statement.
- Consider how the context of a variable defines its value.

Lesson 4: Computational Focus: Conditional Statements

Scientific Context: Mathematical Modeling

Physical Fundamentals: Quadratic Equation

Learning Outcomes:

- Create conditional statements using boolean expressions.
- Write programs with a branching structure using an if statement.

Lesson 5: Computational Focus: For Loops

Scientific Context: Numerical Analysis

Physical Fundamentals: Bisection Method

Learning Outcomes:

- Create and reference individual items of a list.
- Use a for loop to repeat a series of statements in a program.
- Use the range function to create a list of numbers.

Lesson 6 & 7:

Scientific Context: Iteration

Physical Fundamentals: Newton's Method

Lesson 6: Computational Focus: While Loops

Learning Outcomes:

- Swap and increment the values of variables.
- Use a while loop to repeat a series of statements in a program.
- Recognize an infinite loop.

Lesson 7: Computational Focus: Debugging

Learning Outcomes:

- Write Boolean expressions using logical operators not, and, or.
- Use test cases and print statements to track down program errors.

Unit II. Programming Design*Lesson 8 & 9:*

Scientific Context: Describing Data

Physical Fundamentals: Statistics

Lesson 8: Computational Focus: Pseudocode

Learning Outcomes:

- Recognize appropriate program structure to solve problems.
- Write out potential algorithms as pseudocode.
- Use nested loops to repeat an inner loop multiple times.

Lesson 9: Computational Focus: List Methods

Learning Outcomes:

- Use common Python list methods.
- Write code that allows the user to control how many times a loop repeats.

Lesson 10: Computational Focus: Strings

Scientific Context: Bioinformatics & Computational Biology

Physical Fundamentals: Protein & Nucleic Acid Structure

Learning Outcomes:

- Use various string operations and methods.
- Turn a string into a list and then back into a string.

Lesson 11: Computational Focus: Recursion

Scientific Context: Combinatorics

Physical Fundamentals: Multiplicity

Learning Outcomes:

- Understand how recursion works.
- Identify the base case and the recursive call in a recursive function.
- Create a recursive procedure with a function call to itself.
- Recognize when to use recursion and when to use iteration (loops).

Lesson 12 & 13:

Scientific Context: Molecular databases

Physical Fundamentals: Molecular data files

Lesson 12: Computational Focus: File input/output

Learning Outcomes:

- Create a program to read input from a file and write output data to a file.
- Output formatted sequences.

Lesson 13: Computational Focus: Dictionary

Learning Outcomes:

- Use Python dictionaries to simplify coding.

Lesson 14: Computational Focus: Object-oriented programming

Scientific Context: Planetary Motion

Physical Fundamentals: Astronomical Units

Learning Outcomes:

- Understand how to define a class with variables, constructors, and methods.
- Write a method or function with optional arguments.
- Recognize the advantages and disadvantages of using classes.

Unit III. Stochastic Simulations*Lesson 15: Computational Focus: Random Number Sequences*

Scientific Context: Random Processes

Physical Fundamentals: Radioactivity

Learning Outcomes:

- Use Python's random library to generate random numbers and recognize how the seed variable is used when generating a sequence of random numbers.
- Modify the range of a predetermined random number sequence.
- Use list comprehension to initialize a list.
- Create a binning algorithm and plot the results using the Pylab library.

Lesson 16: Computational Focus: Nonuniform Distributions

Scientific Context: Kinetic Molecular Theory

Physical Fundamentals: Maxwell Boltzmann Distribution

Learning Outcomes:

- Use inversion method to create nonuniform distribution of random numbers.
- Use rejection method to create nonuniform distribution of random numbers.

Lesson 17: Computational Focus: Walking Skeleton

Scientific Context: Equilibration

Physical Fundamentals: Gas Diffusion

Learning Outcomes:

- Develop an algorithm to model the diffusion of a gas.
- Implement a walking skeleton when designing a complex program.
- Visually assess when a system reaches equilibrium.

Lesson 18: Computational Focus: Monte Carlo Methods

Scientific Context: Stochastic Simulations

Physical Fundamentals: Monte Carlo Algorithms

Learning Outcomes:

- Implement Monte Carlo method to calculate Π and a nonuniform distribution.

Lesson 19: Computational Focus: Error Analysis

Scientific Context: Uncertainty & Prediction

Physical Fundamentals: Random Walk

Learning Outcomes:

- Use a Monte Carlo algorithm to model the diffusion of a particle.

Unit IV. Deterministic Simulations*Lesson 20: Computational Focus: Arrays*

Scientific Context: Equations of Motion

Physical Fundamentals: 1D Free Fall

Learning Outcomes:

- Use Euler's method to model 1D motion subject to a gravitational force.
- Define/initialize a Python array and use the array in conjunction with various mathematical operations.

Lesson 21: Computational Focus: Module Import

Scientific Context: Truncation Error

Physical Fundamentals: Spring Motion

Learning Outcomes:

- Use Euler's method to model 1D spring motion.

Lesson 22: Computational Focus: Multidimensional Arrays

Scientific Context: Molecular Dynamics

Physical Fundamentals: Modified Euler Methods

Learning Outcomes:

- Compute both energy and position as a function of time for three different integration methods.
- Compare approximate and exact results and analyze the results as a function of time step.

Lesson 23: Computational Focus: Vector Operations

Scientific Context: Molecular Modeling

Physical Fundamentals: Internal Coordinates

Learning Outcomes:

—Use Python vector operations (dot product and cross product) to generate a Z-matrix file from a x,y,z coordinate file for a simple hydrocarbon.

Lesson 24: Computational Focus: Visualization

Scientific Context: 2D Ideal Gas

Physical Fundamentals: Elastic Collisions

Learning Outcomes:

—Generate a simulation of a 2D ideal gas.

—Create graphics using VPython.

ACKNOWLEDGMENTS

The authors would like to thank all our students who took our course for their enthusiasm, feedback, and patience whenever we tried something new. We gratefully acknowledge David Weinstein, and our TOCE reviewers and editors for their comments and suggestions on an earlier version of this article.

REFERENCES

- ABERNETHY, K., GABBERT, P., AND TREU, K. 1998. Inquiry-based computer science instruction: Some initial experiences. *SIGCSE Bull.* 30, 3, 14–17.
- ABRAHAM, M. 2005. Inquiry and the learning cycle approach. In *Chemists' Guide to Effective Teaching*. Vol. 1. Prentice Hall, Upper Saddle River, NJ, 41–52.
- BAREISS, R. AND RADLEY, M. 2010. Coaching via cognitive apprenticeship. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE'10)*. ACM Press, New York, 162–166.
- BRAUGHT, G., WAHLS, T., AND EBY, L. 2011. The case for pair programming in the computer science classroom. *ACM Trans. Comput. Educ.* 11, 1, 1–21.
- BOYER, K., LAHTI, W., PHILLIPS, R., WALLIS, M., VOUK, M., AND LESTER, J. 2010. Principles of asking effective questions during student problem solving. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE'10)*. ACM Press, New York, 460–464.
- BROWN, Q., LEE, F., AND ALEJANDRE, S. 2009. Emphasizing soft skills and team development in an educational digital game design course. In *Proceedings of the 4th International Conference on Foundations of Digital Games (FDG'09)*. ACM Press, New York, 240–247.
- CARTER, L. 2011. Ideas for adding soft skills education to service learning and capstone courses for computer science students. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE'11)*. ACM Press, New York, 517–522.
- CARTER, P. 2012. An experience report: On the use of multimedia pre-instruction and just-in-time teaching in a cs1 course. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE'12)*. ACM Press, New York, 361–366.
- CHIEL, H., GILL, J., MCMANUS, J., AND SHAW, K. 2012. Learning biology by recreating and extending mathematical models. *Science*. 336, 6084, 993–994.
- CHIEL, H., MCMANUS, J., AND SHAW, K. 2010. From biology to mathematical models and back: Teaching modeling to biology students, and biology to math and engineering students. *CBE Life Sci. Educ.* 9, 3, 248–265.
- COLEMAN, B. AND LANG, M. 2012. Collaboration across the curriculum: A disciplined approach to developing team skills. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE'12)*. ACM Press, New York, 277–282.
- EBERLEIN, T., KAMPMEIER, J., MINDERHOUT, V., MOOG, R., PLATT, T., VARMA-NELSON, P., AND WHITE, H. 2008. Pedagogies of engagement in science: A comparison of pbl, pogil, and pltl. *Biochem. Mol. Biol. Educ.* 36, 4, 262–273.
- FALKNER, K. AND FALKNER, N. 2012. Integrating communication skills into the computer science curriculum. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE'12)*. ACM Press, New York, 379–384.
- FARRELL, J., MOOG, R., AND SPENCER, J. 1999. A guided inquiry general chemistry course. *J. Chem. Educ.*, 76, 570–574.
- FEKETE, A., KAY, J., KINGSTON, J., AND WIMALARATNE, K. 2000. Supporting reflection in introductory computer science. *SIGCSE Bull.* 32, 1, 144–148.
- HANSON, D. 2006. *Instructor's Guide to Process-Oriented Guided-Inquiry Learning*. Pacific Crest, Lisle, IL. <http://www.pogil.org/resources/implementation>.

- HANSON, D. AND WOLFSKILL, T. 2000. Process workshops - A new model for instruction. *J. Chem. Educ.* 77, 120–130.
- GOODE, J. AND CHAPMAN, G. 2009. *Exploring Computer Science*. Computer Science Equity Alliance, Los Angeles. <http://www.exploringcs.org/>.
- GREENING, T., KAY, J., KINGSTON, J., AND CRAWFORD, K. 1997. Results of a pbl trial in first-year computer science. In *Proceedings of the 2nd Australasian Conference on Computer Science Education (ACSE'97)*. ACM Press, New York, 201–206.
- HOFFMAN, M., DANSILL, T., AND HERSCOVICI, D. 2006. Bridging writing to learn and writing in the discipline in computer science education. *SIGCSE Bull.* 38, 1, 117–121.
- HU, H. AND KUSSMAUL, C. 2012. Promoting student-centered learning with pogil. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE'12)*. ACM Press, New York, 579–580.
- HUSS-LEDERMAN, S., CHINN, D., AND SKRENTNY, J. 2008. Serious fun: Peer-led team learning in cs. *SIGCSE Bull.* 40, 1, 330–331.
- KUSSMAUL, C. 2011. Process oriented guided inquiry learning for soft computing. In *Proceedings of the International Conference on Advances in Computing and Communication (ACC'11)*.
- KUSSMAUL, C. 2012. Process oriented guided inquiry learning (pogil) for computer science. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE'12)*. ACM Press, New York, 373–378.
- LASSERRE, P. 2009. Adaptation of team-based learning on a first term programming class. In *Proceedings of the 14th Annual ACM SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE'09)*. ACM Press, New York, 186–190.
- MEDLEY, M. 2007. Inquiry-based learning in cs1. *J. Comput. Sci. Coll.* 23, 2, 209–215.
- MINDERHOUT, V. AND LOERTSHER, J. 2008. Facilitation: The role of the instructor. In *Process-Oriented Guided Inquiry Learning: ACS Symposium Series 994*, R. S. Moog and J. N. Spencer, Eds., American Chemical Society, Washington, DC, 72–84.
- MOOG, R., CREEGAN, F., HANSON, D., SPENCER, J., STRAUMANIS, A., BUNCE, D., AND WOLFSKILL, T. 2008. POGIL: Process-oriented guided-inquiry learning. In *Chemists' Guide to Effective Teaching*. Vol. 2., Prentice Hall, Upper Saddle River, NJ, 90–107.
- MOOG, R. AND SPENCER, J. 2008. POGIL: An overview. In *Process-Oriented Guided Inquiry Learning: ACS Symposium Series 994*, R. S. Moog and J. N. Spencer, Eds., American Chemical Society, Washington, DC, 1–13.
- RUDER, S. AND HUNNICUTT, S. 2008. POGIL in chemistry courses at a large urban university: A case study. In *Process-Oriented Guided Inquiry Learning: ACS Symposium Series 994*, R. S. Moog and J. N. Spencer, Eds., American Chemical Society, Washington, DC, 133–147.
- RURSCH, J., LUSE, A., AND JACOBSON, D. 2010. IT-adventures – A program to spark its interest in high school students using inquiry-based learning with cyber defense, game design, and robotics. *IEEE Trans. Educ.* 53, 1, 71–79.
- SIMON, B., KOHANFARS, M., LEE, J., TAMAYO, K., AND CUTTS, Q. 2010. Experience report: Peer instruction in introductory computing. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE'10)*. ACM Press, New York, 341–345.
- STRAUMANIS, A. AND SIMONS, E. 2008. A multi-institutional assessment of the use of POGIL in organic chemistry. In *Process-Oriented Guided Inquiry Learning: ACS Symposium Series 994*, R. S. Moog and J. N. Spencer, Eds., American Chemical Society, Washington, DC, 226–239.
- VANDEGRIFT, T., CARUSO, T., HILL, N., AND SIMON, B. 2011. Experience report: Getting novice programmers to think about improving their software development process. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE'11)*. ACM Press, New York, NY, 493–498.
- VIHAVAINEN, A., PAKSULA, M., LUUKKAINEN, M., AND KURHILA, J. 2011. Extreme apprenticeship method: Key practices and upward scalability. In *Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education (ITiCSE'11)*. ACM Press, New York, 273–277.
- WILLIAMS, L. 2007. Lessons learned from seven years of pair programming at north carolina state university. *SIGCSE Bull.* 39, 4, 79–83.
- YEZISKI, E., BAUER, C., HUNNICUTT, S., HANSON, D., AMARAL, K., AND SCHNEIDER, J. 2008. POGIL implementation in large classes: Strategies for planning, teaching, and management. In *Process-Oriented Guided Inquiry Learning: ACS Symposium Series 994*, R. S. Moog and J. N. Spencer, Eds., American Chemical Society, Washington, DC, 60–71.

Received July 2012; revised March 2013; accepted April 2013